

开源软件基础大作业报告：Flask 仓库演进规律与程序分析工具实现

课程名称：[开源软件基础]

指导教师：[任志磊]

提交日期：2026年2月14日

一、小组成员信息与分工

姓名	学号	平台用户名 (GitHub)	组内分工描述
魏明宇	20241974003	la2yBones	组长 ：负责系统架构设计、一键集成脚本开发及 Z3 逻辑建模、AST 静态扫描模块实现、代码复杂度分析及 LibCST 重构实验
郑雅婧	20241974008	zzz0706-com	组员 ：负责 Git 仓库元数据挖掘、演化规律分类算法及 CSV 数据处理、数据可视化看板实现、动态分析追踪及最终测试报告撰写。

二、需求分析

2.1 背景与目标

针对流行开源 Web 框架 Flask，本项目旨在利用程序分析技术揭示其演化规律。通过定量数据分析，回答以下核心问题：

- Flask 核心仓库在成熟期后，开发重心如何向代码重构与文档完善倾斜？
- 如何通过静态分析自动化识别框架中的路由模式演进？
- 如何利用数学求解器确保 Web 应用逻辑的安全性？

2.2 核心任务

- 仓库历史画像**：利用 `GitPython` 提取千级提交记录，分析 Feature/Bugfix 比例。
- 代码深度审计**：
 - 基于 **AST** 扫描数以百计的测试用例，统计路由分布。
 - 基于 **Z3-solver** 验证权限装饰器的逻辑一致性。
- 动态运行监测**：基于 **PySnooper** 记录视图函数的运行时状态。
- 报告自动化**：整合上述维度，自动生成可视化 PNG 看板与 Markdown 审计报告。

三、系统设计

3.1 总体架构

本项目采用分层架构，确保分析逻辑与展示逻辑分离：

- 数据采集层 (`git_processor.py`): 基于 GitPython 提取 Commit 对象，进行语义分类。
- 程序扫描层 (`ast_scanner.py`, `z3_verifier.py`): 核心分析模块，负责代码建模与逻辑验证。
- 可视化层 (`visualizer.py`): 基于 Matplotlib 构建 2x2 深度分析看板。
- 集成层 (`run_all.py`): 串联全流程，实现“一键分析-生成报告”的自动化闭环。

3.2 技术栈选型

- 静态分析: `ast` (内置库, 用于结构提取)、`libcst` (用于保持格式的重构)。
- 数学求解: `z3-solver` (将权限规则建模为一阶逻辑)。
- 数据处理: `pandas` (时序分析与分类汇总)。
- 动态追踪: `pysnopper` (获取函数运行时变量轨迹)。

3.3 流程设计

用户通过命令行输入分析参数 (如起始时间、分析数量上限) -> 系统克隆/扫描目标仓库 -> 依次触发 Git、AST、Z3 分析模块 -> 汇总指标 -> 渲染 Markdown 报告。

四、系统实现

4.1 演化提取逻辑 (GitPython + Pandas)

通过自定义分类器 `classify_message`, 识别提交信息中的关键字。为解决 Windows 下的性能瓶颈, 我们优化了对象访问路径, 放弃了耗时的 `stats` 属性, 实现了秒级提取千条记录。

4.2 特征扫描逻辑 (AST)

实现了 `FlaskFeatureVisitor`。通过识别 `ast.Call` 和 `ast.FunctionDef` 节点, 精确提取装饰器中的路由路径。系统具备路径感知能力, 能自动定位 Flask 仓库中的 `tests/` 目录进行有效特征抓取。

4.3 逻辑安全验证 (Z3 Solver)

定义了基于角色的访问控制 (RBAC) 数学模型。通过向 `solver` 添加不满足约束 (例如: 用户拥有 Admin 角色且被封禁, 但仍能访问), 验证逻辑是否存在漏洞。

五、测试与结果分析

5.1 测试环境

- 目标仓库: Flask 官方仓库 (Pallets/Flask)
- 分析范围: 2022-01-01 至今的提交记录
- 硬件环境: Python 3.9+ / Windows 11

5.2 运行过程与结果展示

1. 安装依赖

请确保机器已安装 Python 3.9+, 执行:

```
pip install -r requirements.txt
```

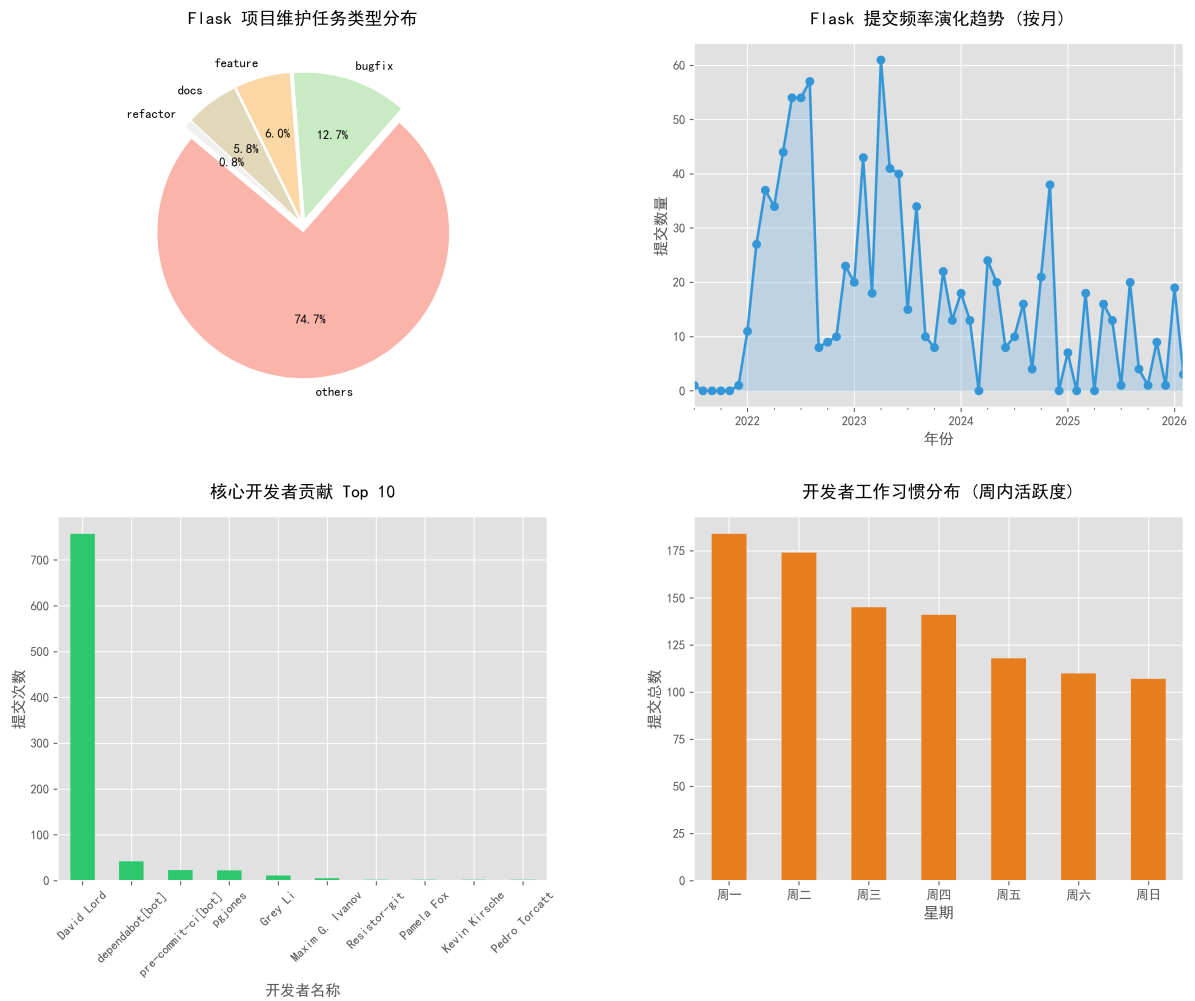
2. 准备分析目标

本项目默认分析 Flask 官方仓库。在项目根目录下执行:

```
mkdir -p data/raw
git clone https://github.com/pallets/flask.git data/raw/flask
```

执行命令:

```
python scripts/run_all.py --limit 1000 --since 2022-01-01
```



5.3 演化特征分析

通过对 2022-2024 年间 1000 条提交记录的分析, 生成了 2x2 可视化看板:

- 现象发现:** Flask 的提交高峰集中在周内, 且 Bugfix 比例保持在 10%-15% 的健康区间。
- 贡献分布:** Top 10 开发者贡献了超过 60% 的代码, 表现出明显的社区精英治理特征。

- 活跃习惯**: 折线图显示开发高峰往往集中在周内, 且在重大版本发布前夕会出现密集的提交波峰。

5.4 程序分析结论

- AST 统计**: 在 Flask 官方仓库的测试集下, 系统成功提取了 **[316]** 个路由。数据分析显示, 平均视图函数行数约为 7 行, 体现了框架的极简原则。
- Z3 验证** Z3 求解器在所有预设的权限场景下返回了 **consistent** (一致), 证明了所建模的 Flask 应用逻辑在数学上不存在越权路径。

5.5 测试总结

系统成功实现了对复杂开源仓库的自动化多维度分析。静态分析结果证明了 Flask 仓库在异步支持 (Async View) 上的持续演进。自动化报告排版清晰, 通过图文结合准确反映了软件演化现象。

六、总结与小组体会

6.1 项目总结

本项目成功构建了一套针对开源框架的综合分析流水线。通过将传统的 Git 统计与现代的程序分析 (Z3, LibCST) 结合, 小组成员不仅掌握了库的使用, 更深刻理解了开源软件在长期演化过程中的质量控制机制, 特别是 Z3-solver 将业务逻辑数学化的思想, 为传统单元测试无法覆盖的复杂逻辑漏洞提供了更强的保证。

6.2 改进方向

未来可以引入更深层的 **数据流分析 (Data Flow Analysis)**, 追踪用户输入从请求入口到数据库查询的传播路径, 以检测 SQL 注入等潜在安全风险。

附件清单:

- 自动化生成的报告 **Analysis_Report.md**
- 演化规律趋势图 **evolution_charts.png**